AI Music Studio 3.0

Group report: training an LSTM to continue music, then turning it into a remix

Yellow boxes like [this] are things your group must check, fill in or change before you submit. Everything else comes from your real notebook results.

Item	Details
Course / lecturer	[course name and lecturer]
Group	Group 25 (Team Fully Charged) [check]
Access number used for the variant	B35448 (this one number decides your variant)
Variant ID / music family	M-01 / Piano phrases (pattern set 1)
Random seed	528376160 (fixed, so results can be repeated)
Assigned investigation	Hidden-state size: 16 units versus 48 units
Date	[submission date]

Group members

No.	Full name	Access number	Main job
1	[name]	B35448	[job]
2	[name]	[number]	[job]
3	[name]	[number]	[job]
4	[name]	[number]	[job]
5	[name]	[number]	[job]

Summary in one minute

We turned music into numbers (one number per note or rest) and trained a small LSTM network to guess the next event. It learned the patterns in our practice data well: its validation loss was **0.436**, against **1.392** for a simple one-step baseline, which is about 69% lower. Our three controlled experiments showed that (1) training for more epochs helps, but with smaller and smaller gains, (2) a bigger hidden state (48 units instead of 16) reduced the loss by about 28%, and (3) sampling temperature changes how closely the model copies patterns it has seen, with no retraining needed. We then asked the LSTM to continue a motif, built a studio arrangement around it, and placed the LSTM melody over a backing track after estimating its tempo and key.

Important honesty point: only the lead melody comes from the LSTM. The bass, chords, drums, beat alignment, key mapping, sound synthesis and mixing are normal rule-based programming. A nicer-sounding track does not prove a better AI.

Where each part of the assignment is answered

What the assignment asks for	Section
Musical representation (events, rests, time steps)	1
LSTM inputs, targets, embedding and memory	2
Baseline and training results	3
Controlled experiment table, graph, observations	4
Personal motif and AI continuation	5
Studio arrangement and production choices	6
Instrumental analysis (waveform, beat, key)	7
Three remix comparisons	8
Failure cases and limitations	9
Learned versus rule-based stages	10
AI-use declaration and files list	11, 12

1. How we represent music as data

A neural network cannot listen to sound. It needs numbers. So in this notebook, music is written as a list of **events**. Every event takes the same small slice of time.

Word	Meaning in our notebook
Event	One step on the timeline. It is either a note or a rest.
Pitch	How high or low a note is. We use MIDI numbers, for example 60 is middle C and 72 is the C one octave higher.
Rest	A silent step. We write it as 0. Silence is also part of the music, so the model must predict it too.
Time step	One event slot. It lasts 0.24 seconds in the raw examples. In the studio mix at 108 BPM, each step is half a beat, about 0.28 seconds.
Token	The whole-number code the model uses for an event, like a word code in text models.
Vocabulary	All possible events. Ours has 9 tokens: the rest plus 8 piano pitches (60, 62, 64, 65, 67, 69, 71, 72).

Our group is in the **Piano phrases** family (variant M-01). The data are synthetic: the notebook builds eight short phrases of 8 steps each, then joins 12 phrases into a track of 96 events, with a small chance (18%) of one random wrong note in a phrase. We made 65 training tracks and 14 validation tracks. Validation tracks are kept apart so we can test the model on music it has never seen.

Two things we noticed after editing one note

- [Write what you changed, for example: we changed one event to 0. Which sound disappeared? What did you see on the piano-roll plot?]
- [Write a second difference you heard or saw.]

This first model predicts only **one event per step**. It does not play chords. Chords and bass are added later by ordinary code (Section 6).

2. How the LSTM works

Input and target. We slide a window along each track. The input is the last **12 events** and the target is the real **next event**. With 96 events per track, each track gives 84 examples, so we had 5,460 training examples and 1,176 validation examples (65 and 14 tracks).

The path through the model: token IDs → embedding → LSTM → output scores → softmax probabilities → next event.

- **Embedding** (12 numbers per token): turns each token ID into a short list of learned numbers. Tokens that behave in similar ways can end up with similar numbers.
- **LSTM** (32 hidden units in our main model): reads the 12 events in order and keeps a memory while it reads.
- **Output layer**: gives one score for each of the 9 possible next events. Softmax turns the scores into probabilities that add up to 1.

Memory. A plain feed-forward network has no memory of earlier steps. An LSTM carries a **hidden state**, a summary of what it has read, and a **cell state** that works like a longer-term notebook. Three learned gates control them: the **forget gate** decides how much old information to keep, the **input gate** decides how much new information to write, and the **output gate** decides what to show as the current hidden state. The gates are learned from data, not written by hand. The LSTM does **not** remember every note forever: it keeps a compressed summary.

Learning. The model guesses the next event. **Cross-entropy loss** measures how wrong the guess was (lower is better). Backpropagation works out how each weight should change, and the **Adam optimiser** changes them (learning rate 0.014). One **epoch** means one full pass through the training data.

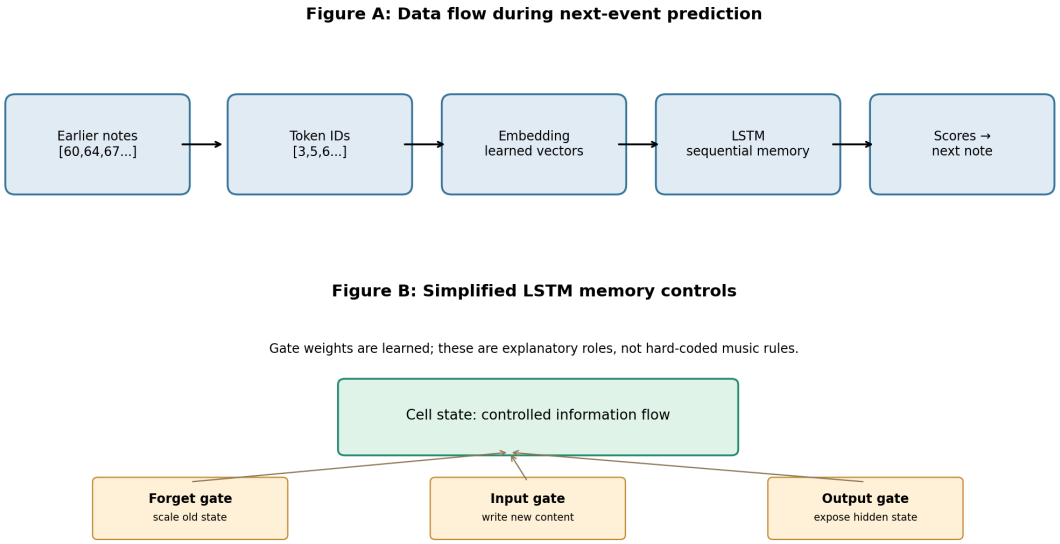


Figure 1. The notebook's teaching diagrams: the token path through the model (top) and the three LSTM gates (bottom).

Generating music. The model predicts probabilities for the next event, we pick one at random using those probabilities, add it to the end, and repeat. This is called autoregressive generation. **Temperature** changes how sharp the probabilities are: a low value makes the likeliest events dominate, a high value lets unlikely events appear more often. Temperature does not make the model smarter.

3. Baseline and training results

Baseline. The baseline looks only at the *current* event and uses a table of how often each event followed each other event in the training tracks. Its validation cross-entropy was **1.392**. This is the number the LSTM has to beat.

LSTM. After 6 epochs (hidden units 32, context 12) the LSTM reached a validation loss of **0.436** and a validation accuracy of **87.5%**. That is about **69% lower loss** than the baseline. The likely reason is that the LSTM sees 12 earlier events, so it can tell which phrase it is inside, while the baseline sees only one.

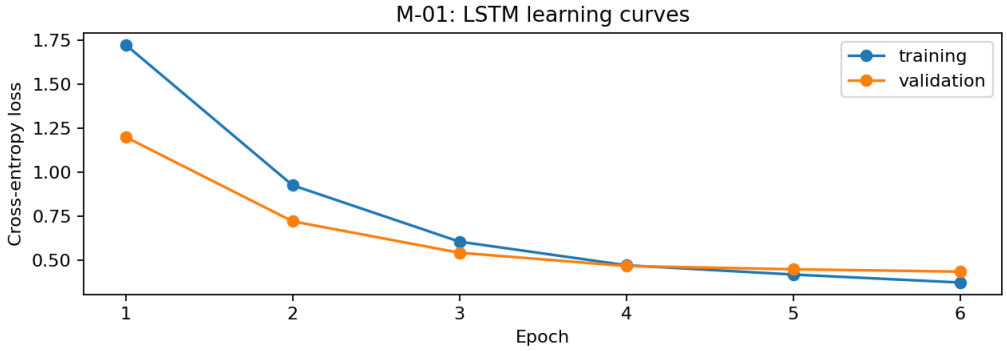


Figure 2. Training and validation loss over 6 epochs.

Reading the curves. Both lines go down quickly in the first 3 epochs, then flatten. Validation loss is *below* training loss in the first epochs because the training number is an average taken while the model is still improving. By epoch 6 the training loss (about 0.37) is lower than the validation loss (about 0.44). A small gap like this is an early sign of overfitting: the model fits the training tracks slightly better than new tracks.

4. Controlled experiments

In a controlled experiment we change **one thing** and keep everything else the same (same seed, same data, same validation set). Training runs were trained separately. The temperature test reuses one fixed trained model.

Condition	Epochs	Validation loss	Validation accuracy
Baseline (one-step transition table)	-	1.392	-
Initial LSTM (hidden 32)	6	0.436	87.5%
Short training (hidden 32)	3	0.543	85.9%
Long training (hidden 32)	8	0.423	87.1%
Assigned test: hidden units = 16	5	0.611	84.7%
Assigned test: hidden units = 48	5	0.437	86.8%

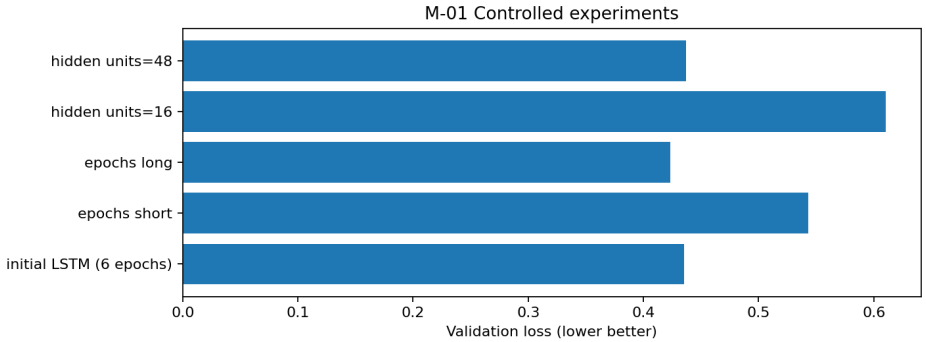


Figure 3. Experiment comparison graph exported by the notebook.

Observations with numbers

- **Experiment 1, training time.** Going from 3 to 6 epochs cut the loss from 0.543 to 0.436 (a drop of 0.108, about 20%). Going from 6 to 8 epochs cut it only from 0.436 to 0.423 (a drop of 0.012, about 3%). More training helps, but the gains get small quickly.
- **Experiment 2, our assigned factor (hidden units).** With 16 units the loss was 0.611 and accuracy 84.7%. With 48 units the loss was 0.437 and accuracy 86.8%. That is a loss drop of 0.174 (about 28%) and 2.1 more percentage points of accuracy. A hidden state with only 16 numbers seems too small to hold the summary the model needs. Note: the 48-unit model at 5 epochs (0.437) is almost the same as the 32-unit model at 6 epochs (0.436), so size and training time can partly make up for each other. We did not run a 32-unit model at exactly 5 epochs, so we cannot separate the two effects exactly.
- **Loss and accuracy do not always agree.** The 6-epoch model has the highest accuracy (87.5%), but the 8-epoch model has the lowest loss (0.423). The differences are small, and each setting was trained only once, so we should not treat tiny gaps as proof.

Which run would we choose?

For next-event prediction we would choose the **8-epoch model (loss 0.423)** because it has the lowest validation loss and its accuracy (87.1%) is within 0.4 points of the best. The 16-unit model is clearly the weakest. Our generated music used the initial 6-epoch model.

What the metrics cannot tell us

Loss and accuracy measure how well the model guesses the next event on synthetic patterns. They cannot tell us whether the music is pleasant, surprising, or musical. A model that copies the training phrases perfectly would score well but sound dull. Listening is still needed.

Experiment 3: sampling temperature

We used the same trained model and the same 16-event starting phrase, and generated 48 new events at three temperatures.

Track	Temperature	Distinct events in the new part	Rests	Same event twice in a row
A focused	0.55	9	4	0
B balanced	1.00	9	2	0
C exploratory	1.45	9	3	2

- **Counting distinct events did not separate the tracks.** All three used all 9 events, because our vocabulary is small. This is a limit of that measure for small vocabularies.
- **Copying.** At 0.55 and 1.00 the new music begins by repeating the last phrase of the starting motif exactly (67, 71, 60, 71, 69, rest, 62, 72). At 1.45 it jumped to a different learned phrase instead, and it contained the repeated note 72, 72, which does not appear in the clean training phrases. So higher temperature gave more change, but also a small mistake.
- [Add what you heard when you listened to A, B and C. Give one concrete example for each, for example which one sounded most repetitive.]

5. Personal motif and AI continuation

A **motif** is a short musical idea. We wrote an 8-event opening and asked the trained LSTM to continue it for 48 events at temperature 0.9.

Item	Value
Our motif (menu numbers, 0 = rest)	[1, 2, 3, 0, 4, 3, 2, 1] [replace with YOUR pattern if you changed it]
Same motif as MIDI pitches	60, 62, 64, rest, 65, 64, 62, 60
What it sounds like in words	[for example: it goes up three steps, rests, then comes back down]

What the LSTM added. The first events after our motif were 67, 67, 65, 69, 65, rest, rest, 67. The model repeated notes and rests here, which suggests it was unsure, because our motif is not one of the eight phrases it learned. After about 8 to 9 events it slid back into phrases from its training data, for example 67, 71, 60, 71, 69, rest, 62, 72. So the model tends to pull new ideas back toward patterns it already knows.

[Describe your first attempt, what you changed, which sounds repeat, and where you placed a rest.]

6. Studio arrangement and production choices

The studio step builds a longer song from the LSTM material. It repeats and shuffles sections of the generated melody (144 events, about 41 seconds at 108 BPM for the three temperature tracks, and 160 events, about 46 seconds for the personal-motif version). It then adds a bass note and a three-note chord every 8 steps, drums (kick, snare, hi-hat), a fade in and out, and a small echo. **All of that is rule-based code.** It is not extra output from the LSTM.

Alternate mix (required)

The assignment asks for at least two production changes, an exported alternate mix, and an explanation. The original mix is kept as evidence.

Setting	Original	Changed to	What we heard
STUDIO_BPM	108	[e.g. 125]	[faster/slower? more energy?]
[BASS_VOLUME or DRUM_VOLUME]	[value]	[value]	[what changed?]

Listening check (at least 3 listeners, one concrete observation each)

Listener	Raw LSTM (1-5)	Arranged (1-5)	One concrete observation
[name]	[score]	[score]	[what did they hear?]
[name]	[score]	[score]	[what did they hear?]
[name]	[score]	[score]	[what did they hear?]

Critical question: did the improvement come from the LSTM or from the arrangement? The melody notes are the same in the raw and arranged versions. What changed is the regular beat, bass, chords and fade, which all come from rules. So a better-sounding arrangement mostly shows good sound design, not a stronger language model.

7. Instrumental analysis: waveform, beat and key

We used the notebook's built-in royalty-free backing track (41 seconds), made from the bass, chords and drums of our balanced arrangement without the LSTM lead. [If you uploaded your own instrumental, say what it is and that you have permission to use it.]

Why an audio recording is not the same as MIDI. MIDI says which note starts when, so each note is easy to use as a token. An MP3 or WAV file is a long list of sound-pressure values (16,000 per second here). We cannot treat each value as a note token, because one note is made of thousands of values and many instruments are mixed together.

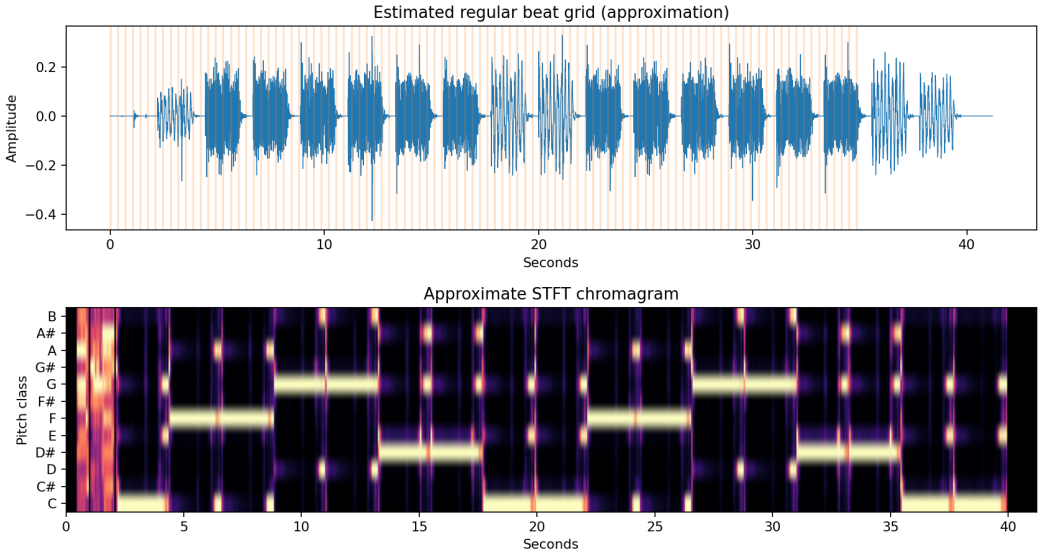


Figure 4. Top: waveform with the estimated beat grid. Bottom: chromagram (energy in each of the 12 pitch classes).

- **Waveform.** It shows loudness over time. Tall peaks are not automatically beats.
- **Tempo.** The beat lines in Figure 4 are about 0.55 seconds apart, which is about 108 beats per minute. This matches the 108 BPM we used to build the backing. [Replace with the exact BPM printed by the notebook.]
- **Chromagram and key.** The bright bars are strongest at C, F, G and E-flat (D#) and repeat in that order. These are the chord roots made by our own bass rule. So the likely key is **C**, and because E-flat and B-flat are used it sounds minor. [Replace with the key and mode printed by the notebook.]

Why these estimates can be wrong. (1) Drums add noise to every pitch class: in the first two seconds of Figure 4 the chromagram is bright across all notes because only drum sound is playing. (2) The chords change every 16 steps, so one key label hides the changes. (3) C minor and E-flat major share the same notes, so the computer can confuse them. (4) The beat grid in Figure 4 stops at about 35 seconds although the audio continues to about 40 seconds, so the tempo tool is only an approximation.

8. Three remix comparisons

For the remix, the LSTM lead melody is placed over the backing track. The notebook lines the notes up with the estimated beat and moves each note to the nearest note of the estimated key's scale. We made three remixes, one for each temperature, and one unconstrained version that skips the scale step.

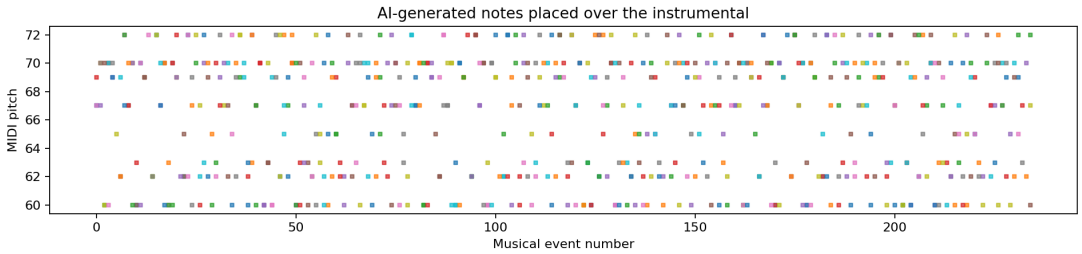


Figure 5. LSTM notes placed over the instrumental after being mapped into the key's scale.

Remix	Temperature	Distinct pitches	What we heard	Rating
A focused	0.55	[from notebook]	[describe]	[1-5]
B balanced	1.00	[from notebook]	[describe]	[1-5]
C exploratory	1.45	[from notebook]	[describe]	[1-5]
D unconstrained (no scale mapping)	1.00	[from notebook]	[sounds more or less in tune with the backing?]	[1-5]

Best remix and why: [Choose one and give a reason that uses what you heard and the numbers above.]

Greatest improvement: [for example: going from raw LSTM to remix made the track feel steady because it now sits on the beat.]

9. Failure cases and limitations

- **Copying and repetition.** At low and medium temperature the model repeated a phrase from the starting motif exactly. The data are synthetic and built from only 8 phrases, so the model has little else to learn.
- **Weak with new ideas.** Our own motif was not in the training data. The model hesitated (repeated notes and rests) and then returned to familiar phrases.
- **Higher temperature adds mistakes, not only variety.** The repeated 72, 72 at 1.45 does not appear in clean training phrases.
- **Small, simple data and a single run per setting.** Differences of one or two tenths of a percent should not be treated as proof.
- **Beat and key tools are approximate.** The beat grid stopped before the end of the audio, and key estimation is unreliable with drums and changing chords.
- **Scale mapping can change the melody.** Moving notes to the nearest scale note makes them fit the backing, but it also changes what the LSTM actually produced.

10. What was learned and what was rule-based

Stage	Learned by the LSTM?	How it works
Predicting the next lead event	**Yes**	Embedding and LSTM weights learned from training tracks
Choosing the actual event	No (random choice)	Random pick using the model's probabilities and the temperature
Training data	No	Synthetic phrases made by code with a little random noise
Song structure (repeating and shuffling sections)	No	Fixed rules that reuse the generated phrase
Bass, chords, drums	No	Rules: bass and chord every 8 steps, chord roots 0, 5, 7, 3 semitones apart; kick, snare and hi-hat patterns
Sound synthesis, fade, echo, mixing	No	Sine-wave based sound code and volume settings
Beat tracking, key estimation	No	Standard audio-analysis methods, not trained by us
Scale mapping and beat alignment	No	Rule: move each note to the nearest scale note and place it on the beat grid

11. AI-use declaration

We used the notebook provided by our lecturer to train the model and make the audio. We also used an AI assistant (Claude) to help us understand the assignment, explain the ideas in simple words and draft this report from our own notebook results. We ran the notebook ourselves and checked the numbers. [Edit this paragraph so it is true for your group, and list any other AI tools or code sources you used.]

The backing track is the notebook's built-in synthetic sample. [If you used your own audio or MIDI, state that you have permission to share it.]

12. Files in the submission package

The ZIP folder contains: the raw AI melody (*M-01_raw_AI_only.wav*); the three studio compositions (*M-01_A_focused_STUDIO*, *B_balanced_STUDIO*, *C_exploratory_STUDIO*, each as WAV and MIDI); the personal motif continuation and studio version (*M-01_personal_continuation*, *M-01_personal_STUDIO*); the built-in instrumental (*M-01_BUILTIN_INSTRUMENTAL.wav*); the three remixes plus the unconstrained version; the figures (*training_curve.png*, *experiment_graph.png*, *audio_beat_chroma.png*, *remix_notes.png*, *lstm_teaching_diagrams.png*) and the table *experiment_comparison.csv*. [Add the alternate mix file and a short README, and remove M-01_temporary_fullmix.wav.]